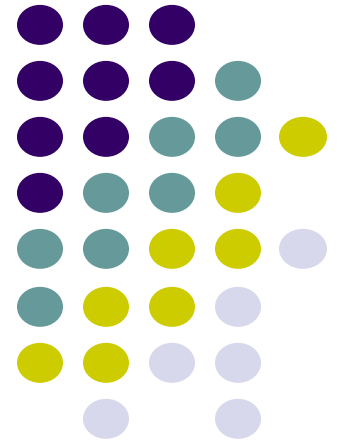
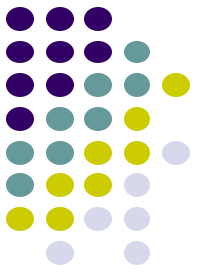


ErSE222: Machine learning in Geoscience

April. 20th, 2025

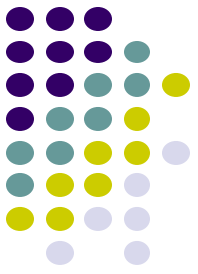
Tariq Alkhalifah and Omar Saad



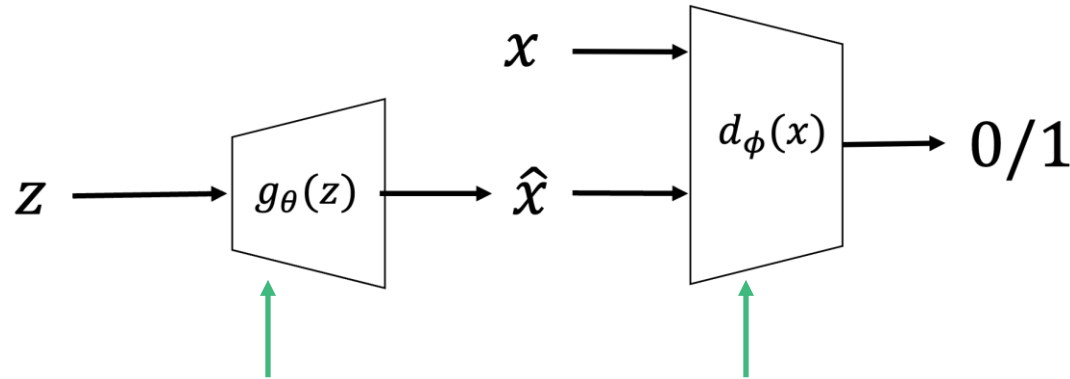


Let's Play a Game!

Generative Adversarial Networks



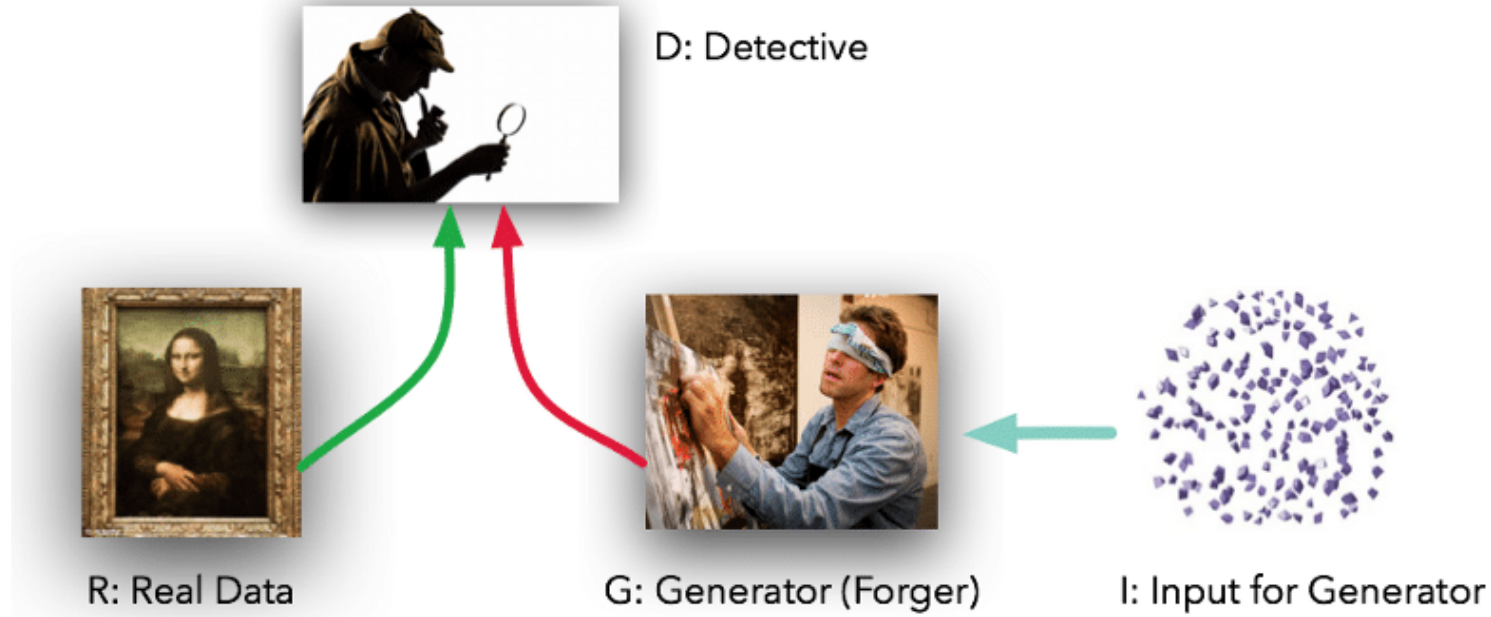
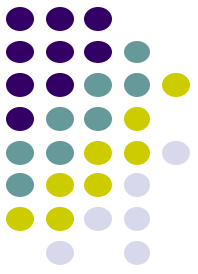
*Family of implicit generative models (do not explicitly estimate a density function), that are composed of two neural networks competing with each other – **adversarial training** (or 2-player min-max game)*



Generator: create a sample of the distribution of the training data from a noise realization z .

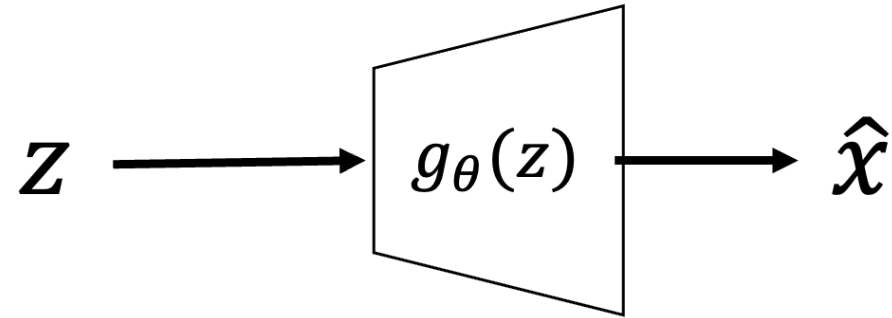
Discriminator: simple classifier tasked to tell if input is true or generated

Generative Adversarial Networks



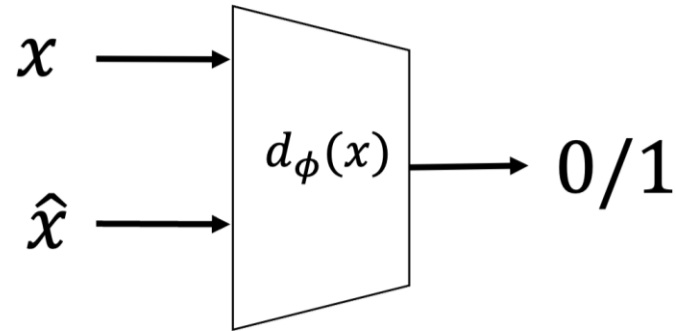
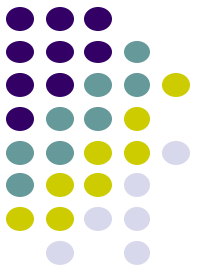
Source: <https://www.novatec-gmbh.de/en/blog/gans-applied-on-novatec-employees/>

GANs Generator



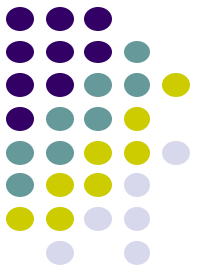
- $\mathbf{z} \in \mathbb{R}^{n_z}$ (with each element sampled from $\mathcal{N} \sim (0,1)$)
- $\hat{\mathbf{x}} \in \mathbb{R}^{n_x}$ created by feeding the vector $\mathbf{z}$ into the generator network
- $\mathbf{z}$ and $\hat{\mathbf{x}}$ can be 1-dimensional, 2-dimensional, or more. Sometimes $\mathbf{z}$ can be of the same dimension of $\hat{\mathbf{x}}$, but sometimes the network can bring a 1d array into a 2d array (common for images in the early days of GANs).
- **No direct training of generator**, gets an indirect feedback from discriminator.

GANs Discriminator

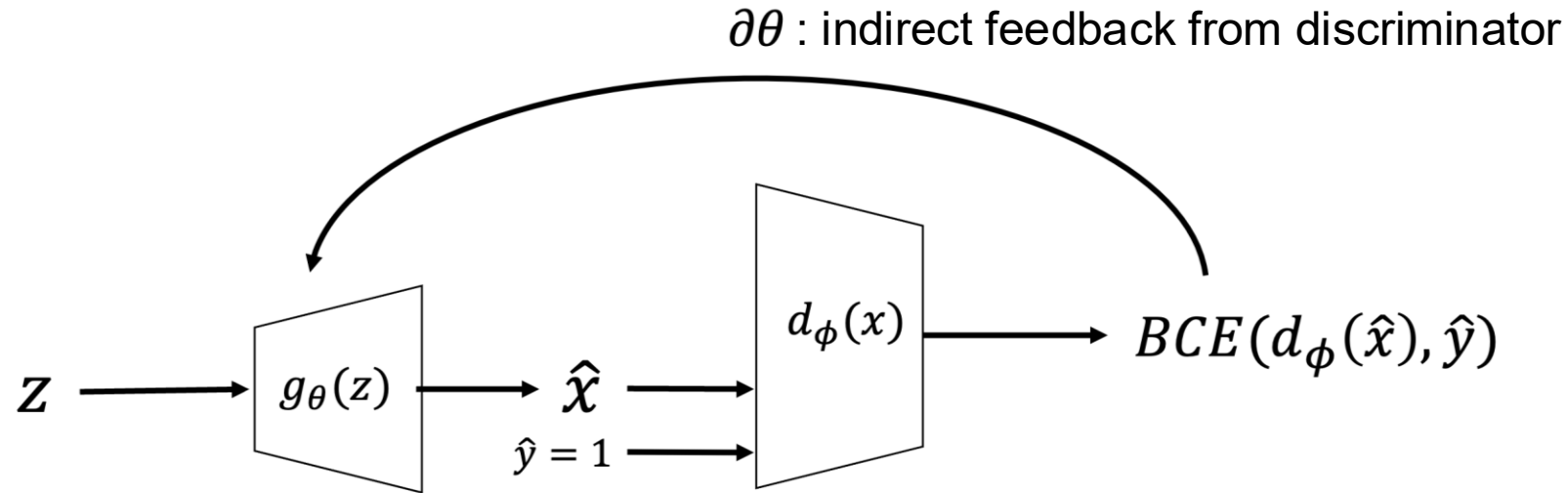


- $\hat{x} \in \mathbb{R}^{n_x}$ created by feeding the vector z into the generator network
- $x \in \mathbb{R}^{n_x}$ sample of the training dataset, from which we want to learn the underlying probability density function ($p(x)$).
- **Discriminator** is trained as **binary classifier** (i.e., BCE loss). Its goal is to tell apart fake ($\hat{x}$) from true (x) samples.

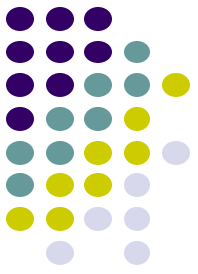
GANs Training process



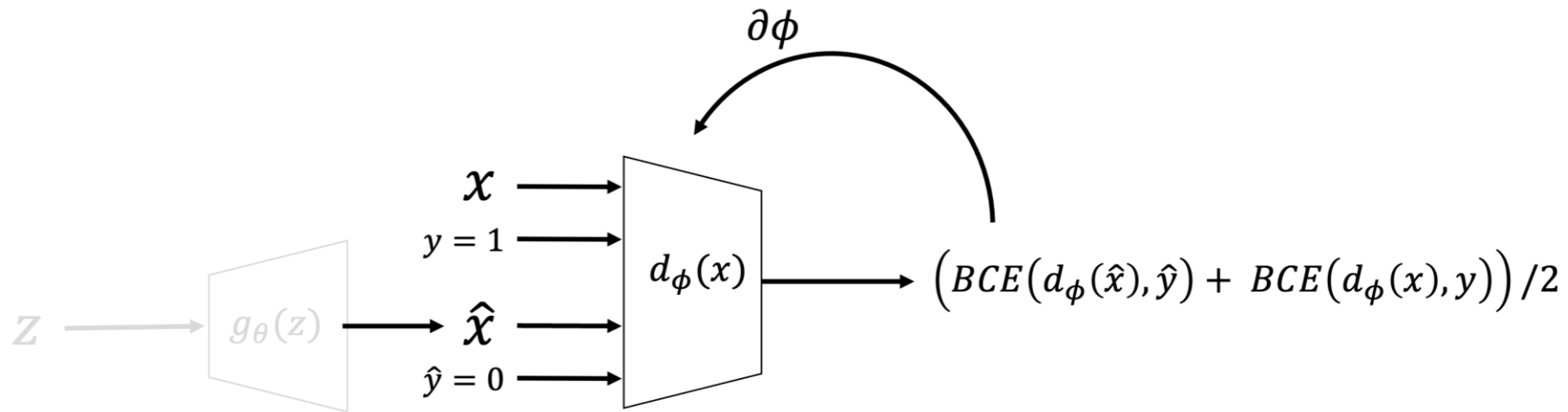
Training is performed in two phases: **generator phase**



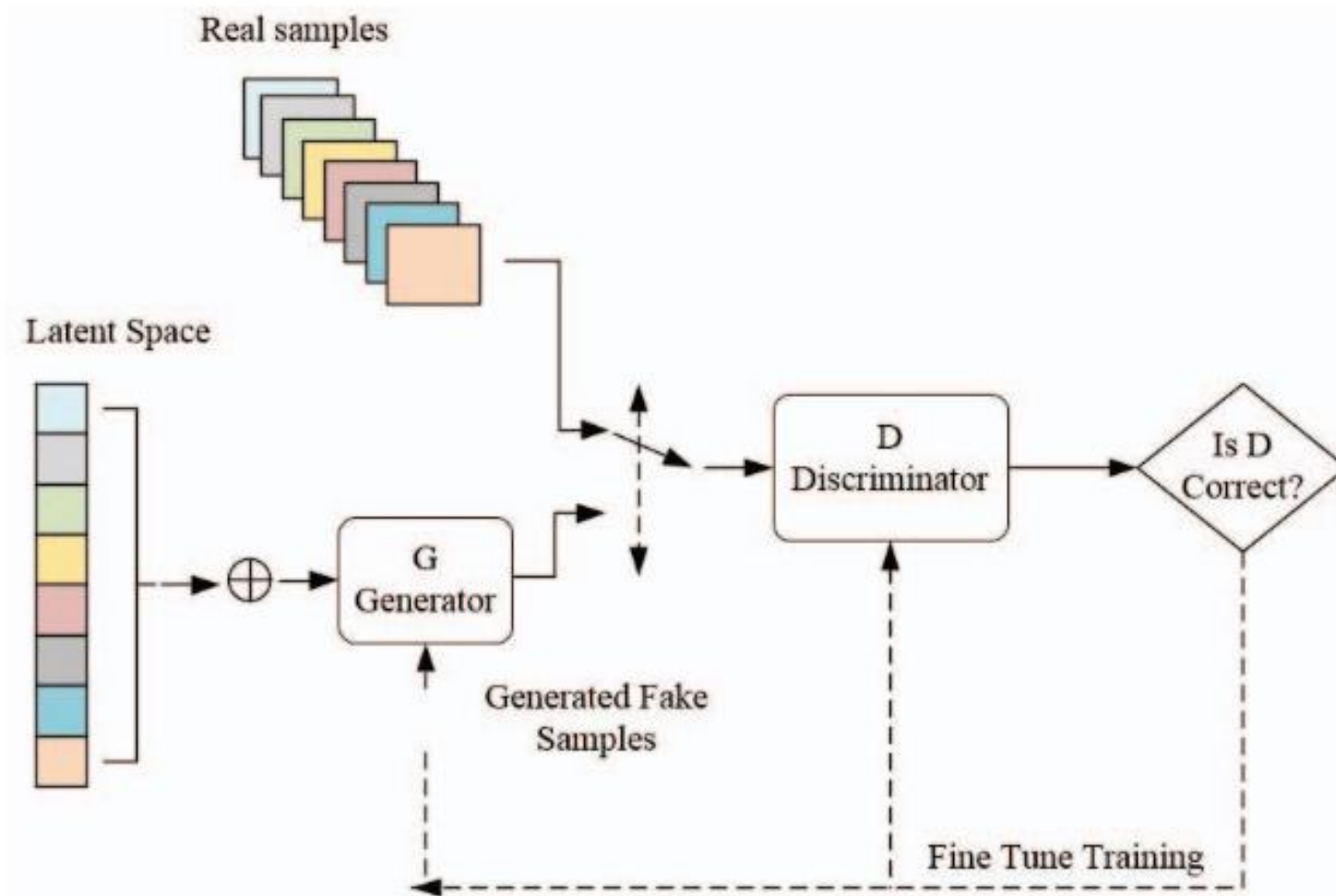
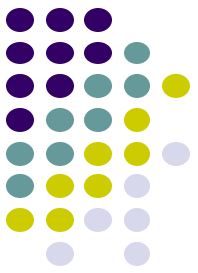
GANs Training process



Training is performed in two phases: **discriminator phase**

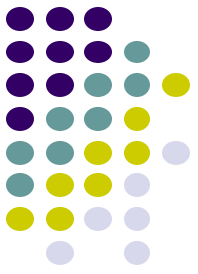


GANs Training process



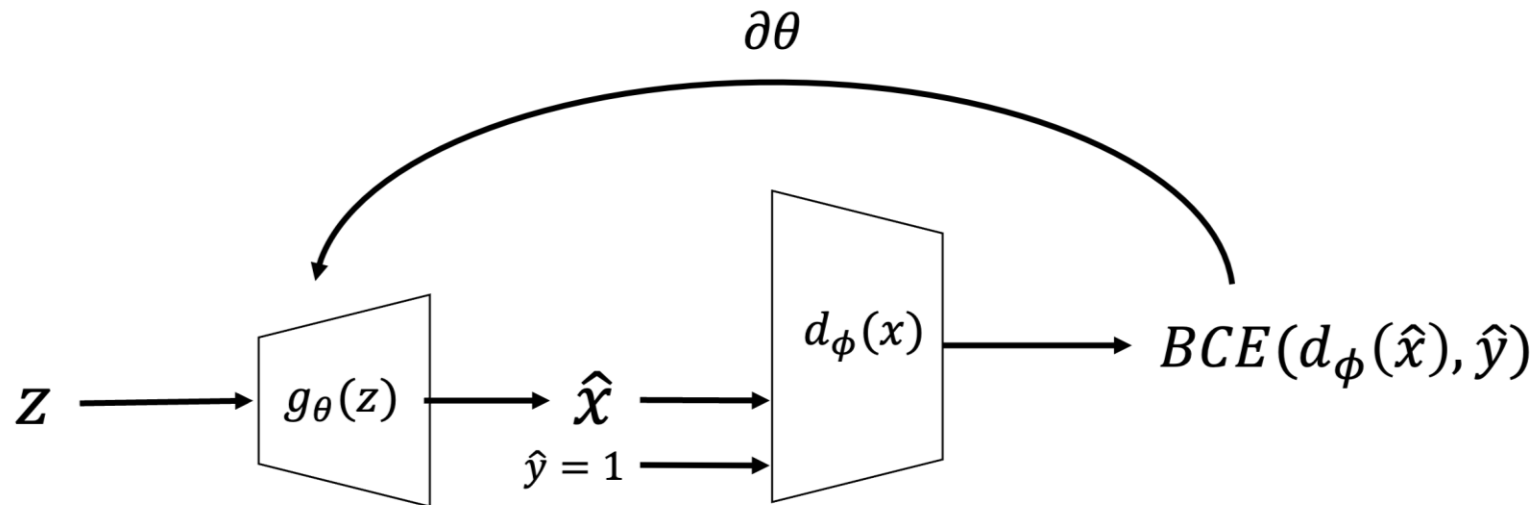
Gonog, L., & Zhou, Y. (2019, June). A review: generative adversarial networks. In *2019 14th IEEE conference on industrial electronics and applications (ICIEA)* (pp. 505-510). IEEE.

GANs Training process

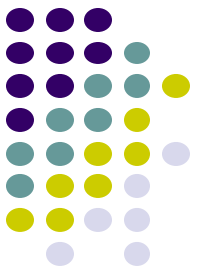


The generator is trying to minimize the following loss function:

$$\mathcal{L}(\hat{\mathbf{x}}, \hat{\mathbf{y}} = 1) = BCE(d_{\phi}(g_{\theta}(\mathbf{z})), \hat{\mathbf{y}} = 1) = -\log(d_{\phi}(g_{\theta}(\mathbf{z})))$$



GANs Training process



The generator is trying to minimize the following loss function:

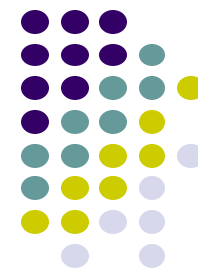
$$\mathcal{L}(\hat{\mathbf{x}}, \hat{\mathbf{y}} = 1) = BCE(d_{\phi}(g_{\theta}(\mathbf{z})), \hat{\mathbf{y}} = 1) = -\log(d_{\phi}(g_{\theta}(\mathbf{z})))$$

The discriminator is trying to minimize the following sum of loss functions:

$$\mathcal{L}(\hat{\mathbf{x}}, \hat{\mathbf{y}} = 0) = BCE(d_{\phi}(g_{\theta}(\mathbf{z})), \hat{\mathbf{y}} = 0) = -\log(1 - d_{\phi}(g_{\theta}(\mathbf{z})))$$

$$\mathcal{L}(\mathbf{x}, \mathbf{y} = 1) = BCE(d_{\phi}(\mathbf{x}), \mathbf{y} = 1) = -\log(d_{\phi}(\mathbf{x}))$$

GANs Training process



The generator is trying to minimize the following loss function:

$$\mathcal{L}(\hat{\mathbf{x}}, \hat{\mathbf{y}} = 1) = BCE(d_{\phi}(g_{\theta}(\mathbf{z})), \hat{\mathbf{y}} = 1) = -\log(d_{\phi}(g_{\theta}(\mathbf{z})))$$



Min when discriminator is
fooled ($d_{\phi}(g_{\theta}(\mathbf{z})) = 1$)

The discriminator is trying to minimize the following sum of loss functions:

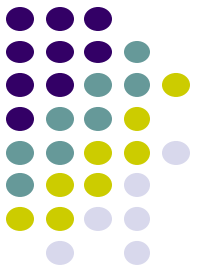
$$\mathcal{L}(\hat{\mathbf{x}}, \hat{\mathbf{y}} = 0) = BCE(d_{\phi}(g_{\theta}(\mathbf{z})), \hat{\mathbf{y}} = 0) = -\log(1 - d_{\phi}(g_{\theta}(\mathbf{z})))$$

$$\mathcal{L}(\mathbf{x}, \mathbf{y} = 1) = BCE(d_{\phi}(\mathbf{x}), \mathbf{y} = 1) = -\log(d_{\phi}(\mathbf{x}))$$



Min when
discriminator is good
($d_{\phi}(g_{\theta}(\mathbf{z})) = 0, d_{\phi}(\mathbf{x}) = 1$)

GANs Training process



A more formal way to write the GAN loss function is via a min-max game:

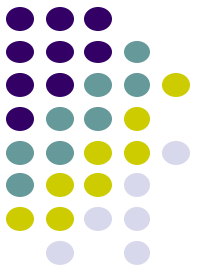
$$\arg \min_{g_{\theta}} \max_{d_{\phi}} \underbrace{E_{\mathbf{x} \sim p_x} [\log(d_{\phi}(\mathbf{x}))]}_{\text{Act on Discriminator output for true data}} + \underbrace{E_{\mathbf{z} \sim p_z} [\log(1 - d_{\phi}(g_{\theta}(\mathbf{z})))]}_{\text{Act on Discriminator output for fake data}}$$

Act on Discriminator
output for true data

Act on Discriminator
output for fake data

*Generator is affected only by second term of loss (wants to maximize it),
Discriminator by both (wants to minimize them).*

GANs in practice



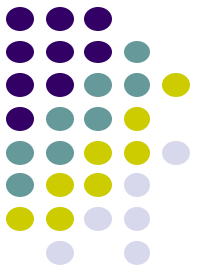
Learning Algorithm

1. Select a batch of training samples and feed them through the discriminator
2. Create a batch of noise samples and feed them through the generator and discriminator
3. Compute the discriminator part of the loss function + backprop.
4. Repeat 1-2-3 N_{disc} times.
5. Create a batch of noise samples and feed them through the generator and discriminator and compute the generator part of the loss function + backprop.

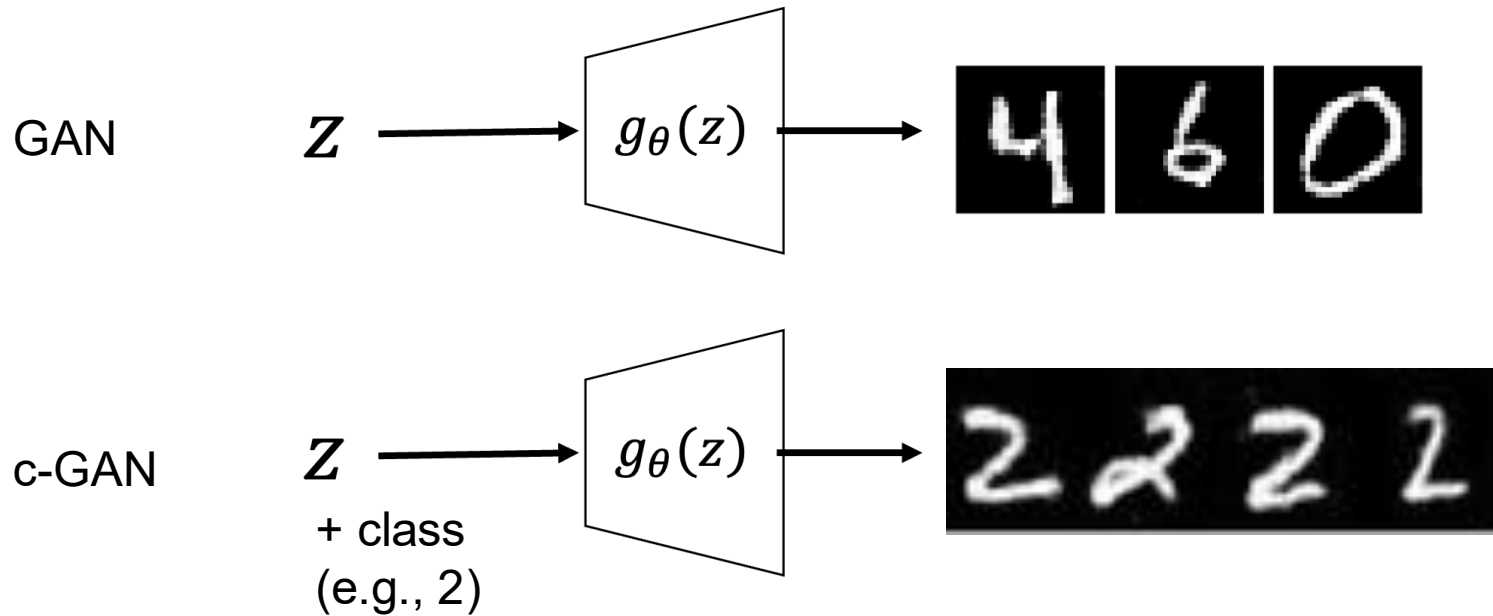
Sampling Algorithm

1. Create a noise sample
2. Feed it into the generator

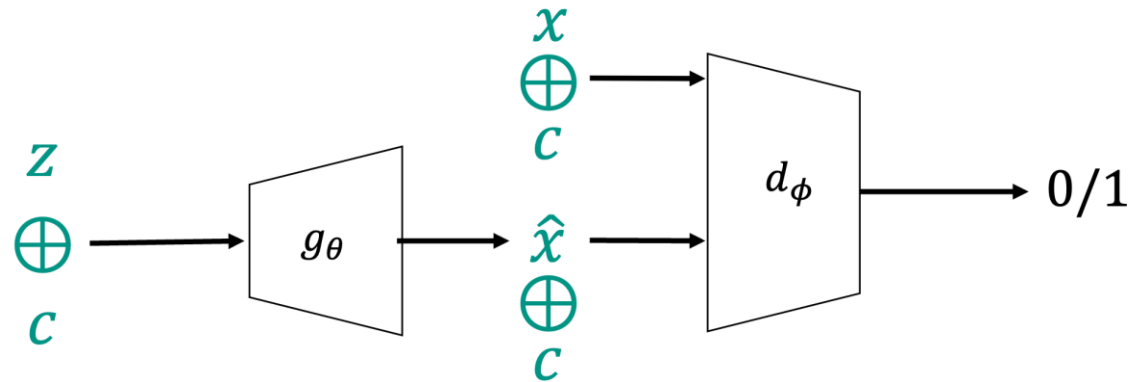
Conditional GANs



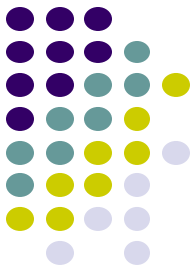
Idea: can we know upfront (before sampling) what we will produce...

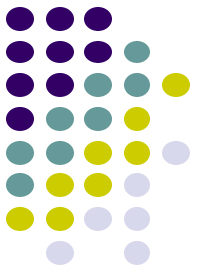


Conditional GANs

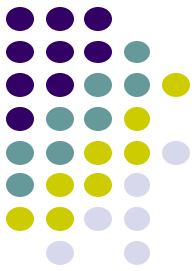
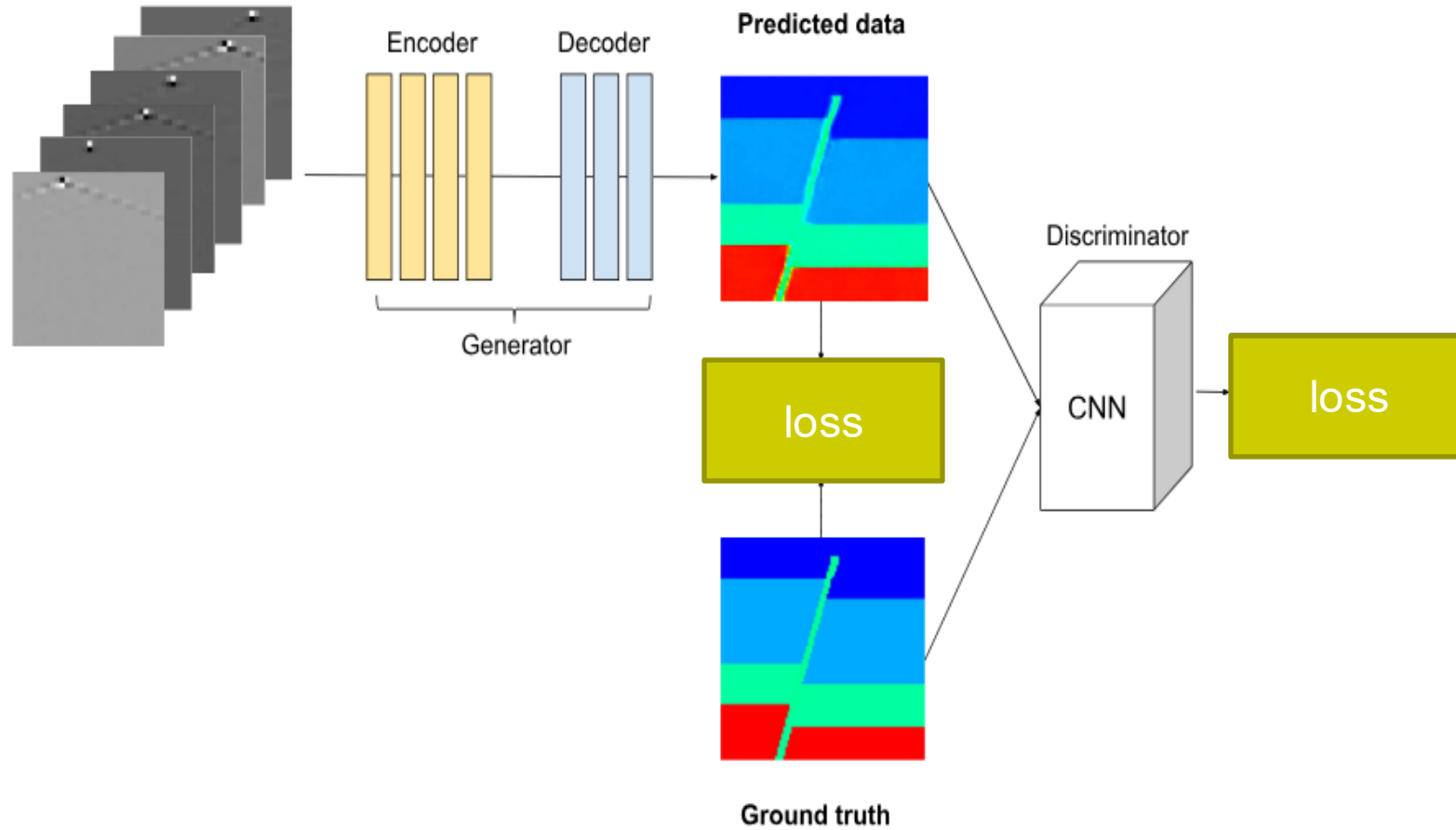


- Concatenate a label c representing a class to random noise ($z \oplus c$), or its one-hot encoded version ($z \oplus c$). Inform generator both at train and generation time.
- Concatenate the same label to x and $\hat{x}$ before feeding them to the discriminator.
- When working with Nd-arrays (e.g., images), concatenation is replaced by channel concatenation (c or c is replicated over the entire 'image').





Applications



Zhang, Z., & Lin, Y. (2020). Data-driven seismic waveform inversion: A study on the robustness and generalization. *IEEE Transactions on Geoscience and Remote sensing*, 58(10), 6900-6913.

